

PyTorch Cheatsheet

深度学习框架常用指令参考 v2.x

张量创建 (Tensor Creation)

从列表数据直接创建张量

```
torch.tensor([1, 2, 3])
```

张量构造器 (默认 float32)

```
torch.Tensor([1, 2, 3])
```

创建全 0 张量 (2行3列)

```
torch.zeros(2, 3)
```

创建全 1 张量

```
torch.ones(2, 3)
```

创建未初始化张量 (占位更快)

```
torch.empty(2, 3)
```

按已有张量形状创建全 0

```
torch.zeros_like(t)
```

按已有张量形状创建全 1

```
torch.ones_like(t)
```

标准正态分布随机数 (均值0, 方差1)

```
torch.randn(2, 3)
```

整数随机张量 (区间 [0, 10))

```
torch.randint(0, 10, (2, 3))
```

范围创建 [start, end) 步长2

```
torch.arange(0, 10, 2)
```

创建全为 9 的张量

```
torch.full((2, 3), 9)
```

属性与转换 (Attributes)

获取张量形状 (Size对象)

```
t.shape / t.size()
```

获取数据类型 (如 torch.float32)

```
t.dtype
```

常用 dtype 常量

```
torch.float32 / torch.bfloat16 / torch.long
```

旧式类型构造器 (不推荐但常见)

```
torch.FloatTensor / torch.LongTensor
```

布尔与字节类型

```
torch.bool / torch.uint8
```

查看所在设备 (cpu / cuda:0)

```
t.device
```

显式创建设备对象

```
torch.device("cuda:0")
```

获取单元素张量的 Python 数值

```
t.item()
```

转 NumPy 数组 (需在CPU上)

```
t.numpy()
```

NumPy 转 Tensor (共享内存)

```
torch.from_numpy(arr)
```

尽量零拷贝贝构造张量 (可指定类型和设备)

```
torch.as_tensor(data, dtype=None, device=None)
```

判断对象是否为 Tensor

```
torch.is_tensor(x)
```

移动到 GPU

```
t.to("cuda") / t.cuda()
```

移动到 CPU

```
t.to("cpu") / t.cpu()
```

类型转换 (float32 / int64)

```
t.float() / t.long()
```

形状操作 (Shape Ops)

改变形状 (数据需内存连续)

```
t.view(shape)
```

改变形状 (通用, 可能会复制)

```
t.reshape(shape)
```

交换两个维度 (转置)

```
t.transpose(0, 1)
```

重新排列所有维度

```
t.permute(0, 2, 1)
```

去除指定的大小为 1 的维度

```
t.squeeze(dim)
```

在指定位置增加大小为 1 的维度

```
t.unsqueeze(dim)
```

沿指定维度拼接张量

```
torch.cat([t1, t2], dim=0)
```

沿新维度堆叠张量

```
torch.stack([t1, t2], dim=0)
```

按指定大小或段数切分

```
torch.split(t, split_size_or_sections, dim=0)
```

平均切分为若干块

```
torch.chunk(t, chunks, dim=0)
```

展平张量 (常用于全连接层前)

```
t.flatten(start_dim=1)
```

数学运算 (Math)

元素加法 (支持广播机制)

```
t1 + t2
```

元素乘法 (Hadamard product)

```
t1 * t2
```

矩阵乘法

```
t1 @ t2 / t1.matmul(t2)
```

求和

```
t.sum(dim, keepdim=False)
```

求平均值

```
t.mean(dim)
```

求最大值 (返回 values 和 indices)

```
t.max(dim)
```

求最大值的索引

```
t.argmax(dim)
```

按条件选择元素

```
torch.where(cond, a, b)
```

张量整体相等判断

```
torch.equal(a, b) / torch.all(a == b)
```

向量或矩阵范数

```
torch.linalg.norm(t, dim=-1)
```

指数与对数

```
torch.exp(t) / torch.log(t)
```

求最小值 (返回 values 和 indices)

```
torch.min(t, dim)
```

双曲正切激活

```
torch.tanh(t)
```

数值截断 (夹逼)

```
torch.clamp(t, min, max)
```

神经网络层 (torch.nn)

自定义模型基类

```
class Net(nn.Module):
```

全连接层 (Linear Layer)

```
nn.Linear(in_ft, out_ft)
```

顺序容器, 快速搭建模型

```
nn.Sequential(...)
```

模块集合容器

```
nn.ModuleList([...]) / nn.ModuleDict({...})
```

可训练参数

```
nn.Parameter(torch.zeros(...))
```

层归一化

```
nn.LayerNorm(hidden)
```

随机失活 (防过拟合)

```
nn.Dropout(p=0.1)
```

常用激活函数

```
nn.GELU() / nn.SiLU() / nn.Tanh() / nn.ReLU()
```

词嵌入层

```
nn.Embedding(num, dim)
```

恒等映射占位层

```
nn.Identity()
```

Transformer 解码层

```
nn.TransformerDecoderLayer(d_model, nhead)
```

单机多卡数据并行

```
nn.DataParallel(model)
```

训练流程 (Training Loop)

定义损失函数 (分类常用)

```
criterion = nn.CrossEntropyLoss()
```

定义优化器 (Adam)

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

定义优化器 (AdamW)

```
optimizer = optim.AdamW(model.parameters(), lr=0.001)
```

学习率调度器

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=1000)
```

1. 清空旧梯度

```
optimizer.zero_grad()
```

2. 前向传播 (Forward)

```
output = model(inputs)
```

3. 计算损失 (Loss)

```
loss = criterion(output, target)
```

4. 反向传播 (Backward)

```
loss.backward()
```

5. 更新参数 (Update)

```
optimizer.step()
```

6. 更新学习率

```
scheduler.step()
```

函数式与损失 (torch.nn.functional)

张量填充 (常用于序列)

```
F.pad(t, (0, pad), value=0)
```

插值上采样

```
F.interpolate(x, scale_factor=2, mode="bilinear")
```

数值稳定的 log-softmax

```
F.log_softmax(logits, dim=-1)
```

交叉熵损失

```
F.cross_entropy(logits, target)
```

均方误差损失

```
F.mse_loss(pred, target)
```

常用激活的函数式接口

```
F.gelu(x) / F.silu(x) / F.relu(x)
```

自动求导 (Autograd)

标记张量需要追踪梯度

```
x.requires_grad = True
```

计算梯度 (默认存入 .grad)

```
y.backward()
```

查看累积的梯度值

```
x.grad
```

上下文: 停止梯度计算 (推理/评估时用)

```
with torch.no_grad():
```

返回一个从计算图分离的新张量 (常用于截断反向传播)

```
x.detach()
```

将带梯度的张量转为 NumPy 数组 (常用组合)

```
x.detach().cpu().numpy()
```

手动清空梯度

```
x.grad.zero_()
```

数据处理 (Data)

继承 Dataset, 实现 __len__, __getitem__

```
class MyDataset(Dataset):
```

构建数据加载器 (批处理、打乱)

```
DataLoader(dataset, batch_size=32, shuffle=True)
```

数据集与加载器工具集

```
torch.utils.data
```

获取一个 Batch 的数据

```
inputs, labels = next(iter(loader))
```

常用图像预处理 (Resize, ToTensor)

```
torchvision.transforms
```

设备与分布式 (Device/Distributed)

检查 GPU 是否可用

```
torch.cuda.is_available()
```

GPU 数量

```
torch.cuda.device_count()
```

当前 GPU 索引

```
torch.cuda.current_device()
```

设置当前进程使用的 GPU

```
torch.cuda.set_device(rank)
```

释放未使用的显存缓存

```
torch.cuda.empty_cache()
```

等待当前设备完成计算

```
torch.cuda.synchronize()
```

获取当前进程 rank

```
torch.distributed.get_rank()
```

获取进程总数

```
torch.distributed.get_world_size()
```

分布式规约同步

```
torch.distributed.all_reduce(t, op=torch.distributed.ReduceOp.SUM)
```

全局同步屏障

```
torch.distributed.barrier()
```

多进程工具 (分布式启动)

```
torch.multiprocessing
```

保存与加载 (Save/Load)

仅保存模型参数 (推荐)

```
torch.save(model.state_dict(), 'w.pth')
```

加载模型参数

```
model1.load_state_dict(torch.load('w.pth'))
```

在模型间复制参数 (常用于目标网络更新)

```
model1.load_state_dict(model2.state_dict())
```

目标网络软更新 (Soft Update)

```
for p_tgt, p in zip(target.parameters(), net.parameters()):
    p_tgt.data.copy_(p_tgt.data * (1 - tau) + p.data * tau)
```

保存整个模型 (结构+参数)

```
torch.save(model, 'model.pth')
```

加载整个模型

```
model = torch.load('model.pth')
```

训练模式 (启用 Dropout/BatchNorm)

```
model.train()
```

评估模式 (禁用 Dropout/BatchNorm)

```
model.eval()
```

概率分布 (Distributions)

导入类别分布 (常用于强化学习离散动作)

```
from torch.distributions import Categorical
```

根据 logits (未归一化概率) 创建分布

```
dist = Categorical(logits=logits)
```

按概率采样动作

```
action = dist.sample()
```

计算动作的对数概率 (actions 必须是 torch.Long)

```
log_probs = dist.log_prob(actions)
```

计算分布的熵 (常用于鼓励探索)

```
entropy = dist.entropy()
```

实用技巧 (Tips)

设置随机种子 (复现性)

```
torch.manual_seed(42)
```

显式生成器控制随机性

```
g = torch.Generator().manual_seed(42)
```

为特定设备创建随机生成器

```
torch.Generator(device="cuda")
```

深拷贝张量 (记录梯度历史)

```
t.clone()
```

设置打印精度

```
torch.set_printoptions(precision=4)
```

维度操作图解

行方向 (向下压缩) / 批次维度

```
dim=0
```

列方向 (向右压缩) / 特征/通道维度

```
dim=1
```

保持聚合后的维度为 1, 便于广播

```
keepdim=True
```

Numpy风格增加维度 (=unsqueeze)

```
None / [None, :]
```